Responding to Injects - Your Opportunity for Success

The response to all injects must be in a PDF document. Other document formats, such as Word or RTF will **not be accepted** and the inject **not scored**. The inject scoring engine is built around the standard of using PDF documents. For some injects you may be asked to supply a second, ZIP file with a particular file, such as a log or packet capture. These are not frequent.

Part of the CCDC scenario is the simulation of a business or other organization. Responses should be of the type that would be used in these organizations. You will need to demonstrate your business communication skills in inject responses –- this is part of the score.

Construct your response in the form of a business memo. Use language and sentence structure that would be used if responding to an executive or management layer individual. These job functions are often not staffed with technically knowledgeable people. The business form and content is part of the inject score.

The memo should concisely (not a lot of words) and succinctly respond to the specifics of the inject. When including tabular, graphical or screenshot content, be sure to explain what the reader is looking at, and why that content is relevant to the response.

The inject response is your opportunity to convince a judge that the team is knowledgeable and skilled in the particular area/topic. The judge is looking for that level of competence and the proof that the goal was really accomplished. When preparing screen shots and other images, be sure they are readable, and the caption/title clearly describes what the image is telling the reader.

The use of AI tools, such as CHAT/GPT to generate responses to injects will result in zero points. The use of these tools as reference materials from which to develop inject responses are the same as using any other reference materials. As in using any reference materials, direct quotation requires attribution in order to avoid plagiarism issues.

Lastly, judges are reading a lot of injects, so to score the most points, it is to the team's advantage to produce a document that quickly guides the reader to the key elements in the response. Much like a resume, if the reader has to analyze the document in order to ascertain what is being described then the score is likely to be lower.

When writing technical installation documents, rather than a business memo. Consider the following framework for ideas to develop a technical document.

Framework for Writing IT Technical Documents (General)

A strong IT technical document is **clear, structured, and actionable**. It must allow a reader—whether a technician, engineer, administrator, or end-user—to perform a task or understand a system without ambiguity.

Below is the recommended structure.

1. Title & Document Control

Every IT document should begin with administrative metadata:

- **Title** (clear and specific)
- **Date created / last updated**
- **Intended audience** (e.g., sysadmins, help desk, end users)

2. Purpose / Overview

Explain **why the document exists**:

- What problem it solves.
- What tasks or systems it relates to.
- Business or operational context (high-level).

3. Scope & Applicability

Define **what is included and what is not**:

- Systems covered.
- Operating systems or platforms.
- Environments (development, test, staging, production).

4. Prerequisites

List **requirements necessary before the reader proceeds**:

- Required skills or roles (e.g., “Administrator rights required”).
- Tools or software needed.
- Network or system preconditions.
- Access permissions, accounts, or credentials.
- Dependencies such as libraries, servers, or runtime environments.

5. Definitions, Acronyms, and References

For clarity:

- Define unusual or domain-specific terms.
- List acronyms and their meanings.
- Provide links to related documents or knowledge bases.

6. Procedures / Core Content

This is the heart of the document.

Guidelines for writing procedures:

- Use **numbered, sequential steps**.
- Make each step **actionable** (“Run this command,” “Open this console”).
- Include **screenshots**, diagrams, command examples, or code snippets as needed.
- Highlight **expected outcomes** after each major step.
- Include **warnings**, **cautions**, or **required verification checks**.

This section may vary in type:

- Installation steps
- Configuration steps

- Maintenance procedures
- System architecture description
- Network configuration
- Back-up/restore processes
- Deployment guides
- Monitoring or alert-handling runbooks
- Security configuration

7. Validation & Verification

After the procedure, instruct how to confirm success:

- Commands to check system status.
- Logs to review.
- Expected metrics, outputs, or behaviors.
- Known indicators of success or failure.

8. Troubleshooting / Common Issues

Provide:

- Frequent errors and symptoms.
- Root causes.
- Step-by-step solutions.
- Log file locations.
- Diagnostic commands or tools.
- Escalation workflow if troubleshooting fails.

9. Security Considerations

If relevant, include:

- Access control requirements.
- Password or credential handling rules.
- Firewall requirements or port rules.
- Compliance requirements (PCI, NIST, HIPAA, etc.).
- Audit and logging requirements.

10. Backup, Recovery, and Rollback (If Applicable)

Describe:

- How to reverse the procedure safely.
- How to restore a previous configuration.
- Backup frequency and method.
- Disaster recovery processes.

11. Appendix / Supporting Materials

Include:

- Configuration file samples.
- Code snippets.
- CLI reference tables.
- Network diagrams.
- Flowcharts.
- Additional resources.

Principles of Effective IT Technical Writing

Clarity

Write plainly. Avoid ambiguity.

Step-by-step precision

Assume the reader must follow steps literally.

Consistency

Uniform formatting, terminology, and structure across all documents.

Audience awareness

Match technical depth to the people performing the task.

Reproducibility

Another person should achieve identical results using your document alone.

Validation

Ensure the document is correct by testing instructions on a clean system/environment.